CHAPTER 6
COMBINATIONAL LOGIC DESIGN

6.1 Introduction

Digital logic circuits may be combinational or sequential. A *combinational* circuit consists of logic gates whose outputs at any time are determined directly from the present combination of inputs without regard to previous inputs. A combinational circuit performs a specific information processing operations fully specified logically by a set of Boolean functions. *Sequential* circuits employ memory element in addition to logic gates. Their outputs are a function of the inputs and the state of the memory elements. The state of memory elements, in turn, is a function of previous inputs. As a consequence, the outputs of a sequential circuit depend not only on present inputs, but also on past inputs, and the circuit behavior must be specified by a time sequence of inputs and internal states. Sequential circuits are discussed in Chapter 10.

The purpose of this chapter is to use the knowledge acquired in previous chapters and formulate systematic design and analysis procedures of combinational circuits.

A combinational circuit consists of input variables, logic gates, and output variables. The logic gates accept signals from the inputs and generate signals to the outputs. This process transforms binary information from the given input data to the required output data. Obviously, both input and output are represented by binary signals, i.e., they exist in two possible values, one representing 1 and the other 0. A block diagram of a combinational circuit is shown in Figure 6.1. The n input binary variables come from an external source; the m output variables go to an external destination. In many applications, the source and/or destination are storage registers (discussed in Chapter 11) located either in the vicinity of the combinational circuit or in a remote external device.

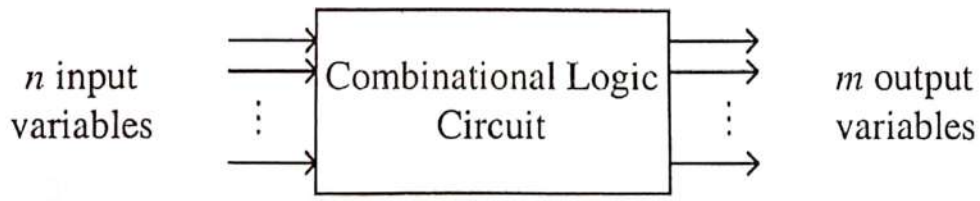


Figure 6.1 Block diagram of a combinational circuit.

For n input variables, there are 2^n possible combinations of binary input values. For each possible input combination, there is one and only one possible output combination. A combinational circuit can be described by m Boolean functions, one for each output variable. Each output function is expressed in terms of the n input variables.

6.2 Design of Combinational Circuits

The design of combinational circuits involves the following steps:

1. The problem is stated.

2. The number of available input variables and required output variables is determined. The input and output variables are assigned letter symbols and logic values.
3. The truth table that defines the required relationships between inputs and outputs is derived.
4. The simplified Boolean function for each output is obtained.
5. The logic diagram is obtained.

A truth table for a combinational circuit consists of input columns and output columns. The binary values for the outputs are determined from examination of the stated problem. The output functions specified in the truth table give the exact definition of the combinational circuit. It is important that the verbal specifications be interpreted correctly into a truth table. Sometimes the designer must use his intuition and experience to arrive at the correct interpretation. Word specifications are very seldom complete and exact. Any wrong interpretation that results in an incorrect truth table produces a combinational circuit that will not fulfil the stated requirements.

The output Boolean functions from the truth table are simplified by any available method, such as algebraic simplification, the K-map method, the Quine-McCluskey method, or the iterated consensus method. Usually there will be a variety of simplified expressions from which to choose. However, in any particular application, certain restrictions, limitations, and criteria will serve as a guide in the process of choosing a particular algebraic expression. A practical design method would have to consider such conditions as

(1) minimum number of gates,
(2) minimum number of inputs to a gate,
(3) minimum propagation time of the signal through the circuit,
(4) minimum number of interconnections, and
(5) limitations of the driving capabilities of each gate.

Since all these criteria can not be satisfied simultaneously, and since the importance of each constraint is dictated by the particular application, it is difficult to make a general statement as to what constitutes an acceptable simplification. In most cases the simplification begins by satisfying an elementary objectives, such as producing a simplified Boolean function in a standard form, and from that proceeds to meet any other performance criteria.

In practice, designers tend to go from the Boolean functions to a wring list that shows the interconnections among various standard logic gates. In that cases the design need not go any further than the required simplified output Boolean functions. However, a logic diagram is helpful for visualizing the gate implementation of the expressions.

The design procedure is demonstrated using the following example.

1. **The problem statement:** The door of an explosive-warehouse is operated on by four persons, namely, *A*, *B*, *C*, and *D*. Person *A* along with at least any one from *B*, *C*, and *D* can open the door. If *A* is absent, *B*, *C*, and *D* can open the

door altogether. A combinational logic circuit is to be designed for controlling the door.

2. Four input lines are required for four keys operated by four persons. The four input variables are assigned letter symbols A, B, C, and D after the names of the four persons. The input variables are assigned the following logic values:

   The key is operated for opening the door: 1
   The key is operated for closing the door: 0

   One output line is required for controlling the lock of the door. The output variable is assigned the letter symbol X and the following logic values:

   The lock is opened: 1
   The lock is closed: 0

3. The truth table defining the relationship between inputs and output is derived and shown in Table 6.1.

Table 6.1 Truth table for explosive-warehouse door control example.

$A\ B\ C\ D$	X
0 0 0 0	0
0 0 0 1	0
0 0 1 0	0
0 0 1 1	0
0 1 0 0	0
0 1 0 1	0
0 1 1 0	0
0 1 1 1	1
1 0 0 0	0
1 0 0 1	1
1 0 1 0	1
1 0 1 1	1
1 1 0 0	1
1 1 0 1	1
1 1 1 0	1
1 1 1 1	1

4. The simplified Boolean function for the truth table of Table 6.1 is obtained by K-map method in Figure 6.2, which gives

$$X = AB + AC + AD + BCD$$

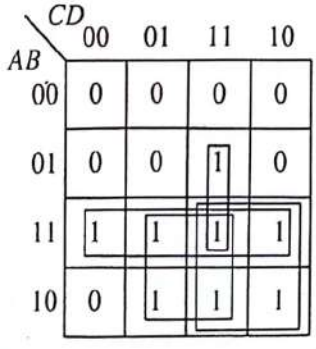


Figure 6.2 Simplification of the function of Table 6.1 by K-map.

5. The logic diagram of the door control circuit is shown in Figure 6.3.

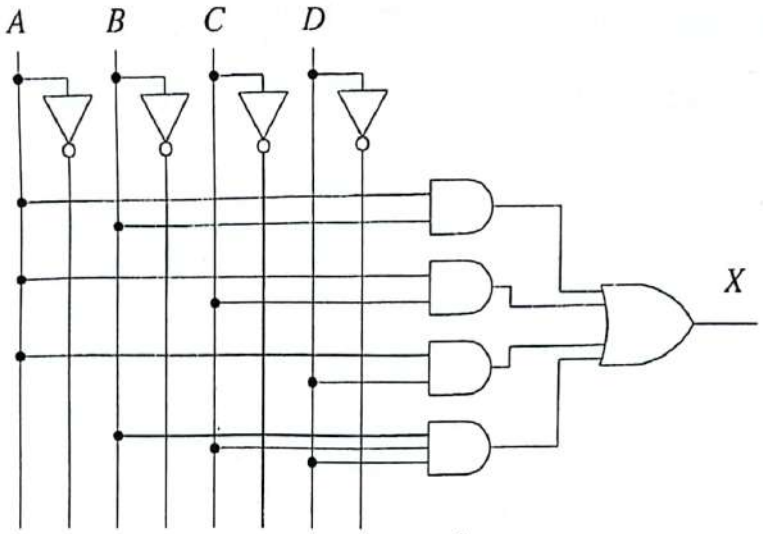


Figure 6.3 Logic diagram for the explosive-warehouse door control example.

The design of some typical elementary functions important for the understanding of digital computers and systems will be discussed in the following chapters.

6.3 Analysis of Combinational Circuits

The design of a combinational circuit starts from the verbal specifications of a required function and culminates with a logic diagram. The analysis of a combinational circuit is somewhat the reverse process. It starts with a given logic diagram and culminates with a set of output Boolean functions or a truth table or a verbal explanation of the circuit operation. If the logic diagram to be analyzed is accompanied by a function name or an explanation of what it is assumed to accomplish, then the analysis problem reduces to a verification of the stated function.

The first step in the analysis is to make sure that the given circuit is combinational and not sequential. The diagram of a combinational circuit has logic gates with no feedback paths or memory elements. A feedback path is a connection from the output of one gate to the input of a second gate that forms part of the input to the first gate. Feedback paths or memory elements in a digital circuit define a sequential circuit and must be analyzed according to procedures outlined in Chapter 10.

Derivation of Boolean Expressions

Once the logic diagram is verified as a combinational circuit, the Boolean expressions can be derived using the following steps:

1. Label with arbitrary symbols all gate outputs that are a function of the external input variables only. Obtain the Boolean function for each gate.
2. Label with other arbitrary symbols those gates that are a function of input variables and/or previously labeled gates. Find the Boolean functions for these gates. Repeat the above process until the outputs of the circuit are obtained.

3. By repeated substitution of previously defined functions, obtain the output Boolean functions in terms of external input variables only.

The procedure is demonstrated below for the example combinational circuit of Figure 6.4.

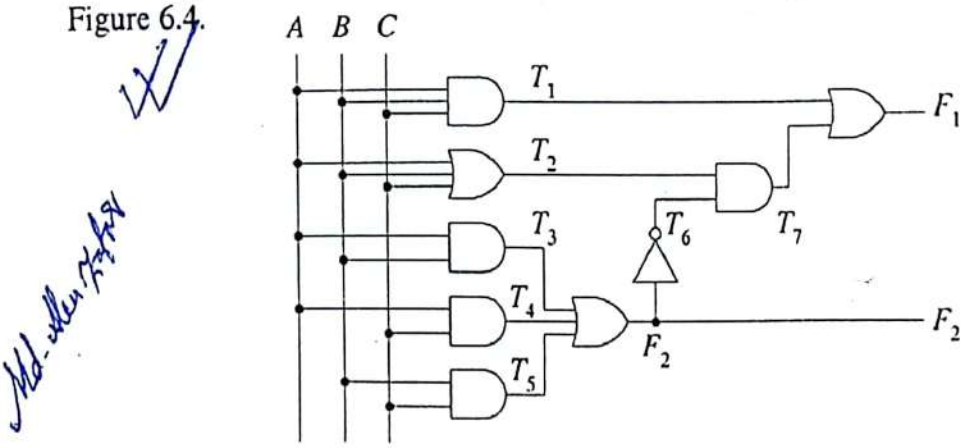


Figure 6.4 Combinational logic circuit for analysis example.

1. The outputs of gates that are a function of external input variables only are T_1, T_2, T_3, T_4, and T_5. The Boolean functions for these five outputs are

$$T_1 = ABC$$
$$T_2 = A + B + C$$
$$T_3 = AB$$
$$T_4 = AC$$
$$T_5 = BC$$

2. Now we consider outputs of gates that are a function of already defined symbols:

$$F_2 = T_3 + T_4 + T_5$$
$$T_6 = F_2'$$
$$T_7 = T_2 T_6$$
$$F_1 = T_1 + T_7$$

3. The output Boolean function F_2 in terms of external input variables only is obtained as follows:

$$F_2 = T_3 + T_4 + T_5 = AB + AC + BC$$

The output Boolean function F_1 in terms of external input variables only is obtained by repeated substitution of previously defined functions as follows:

$$
\begin{aligned}
F_1 &= T_1 + T_7 \\
&= ABC + T_2T_6 \\
&= ABC + (A+B+C)F_2' \\
&= ABC + (A+B+C)(AB+AC+BC)' \\
&= ABC + (A+B+C)(A'+B')(A'+C')(B'+C') \\
&= ABC + (AB'+A'B+A'C+B'C)(A'B'+A'C'+B'C'+C') \\
&= ABC + AB'C' + A'BC' + A'B'C
\end{aligned}
$$

Derivation of Truth Table

Construction of truth table is straightforward from the derived Boolean functions. However, the truth table can be obtained directly from the logic diagram without going through the derivations of the Boolean functions by using the following steps:

1. Determine the number of input variables to the circuit. For n inputs, form the 2^n possible input combinations of 1s and 0s.
2. Label the outputs of gates with arbitrary symbols.
3. Obtain the truth table for the outputs of those gates that are a function of external input variables only.
4. Proceed to obtain the truth table for the outputs of those gates that are a function of previously defined values until the columns for all outputs are determined.

This process is demonstrated using the circuit of Figure 6.4 as follows:

1. The input variables of the circuit are A, B, and C. The $2^3 = 8$ input combinations of 1s and 0's for three variables are shown in the left column of Table 6.2.
2. The gates are labeled as in Figure 6.4.
3. Outputs T_1, T_2, T_3, T_4, and T_5 are functions of external input variables only and they are

$$
\begin{aligned}
T_1 &= ABC \\
T_2 &= A + B + C \\
T_3 &= AB \\
T_4 &= AC \\
T_5 &= BC
\end{aligned}
$$

   Truth table for T_1, T_2, T_3, T_4, and T_5 are derived from input combinations and are shown in the Table 6.2.
4. Outputs F_2, T_6, T_7, and F_1 are function of previously defined values and they are

$$F_2 = T_3 + T_4 + T_5$$
$$T_6 = F_2'$$
$$T_7 = T_2 T_6$$
$$F_1 = T_1 + T_7$$

Truth table for F_2 is derived from the truth table of T_3, T_4, and T_5. Truth table for T_6 is derived from the truth table of F_2. Truth table for T_7 is derived from the truth tables of T_2 and T_6. Finally, truth table for F_1 is derived from the truth tables of T_1 and T_7. These truth tables are shown in Table 6.2.

Table 6.2 Truth table for the operation of the logic diagram of Figure 6.4.

ABC	T_1	T_2	T_3	T_4	T_5	F_2	T_6	T_7	F_1
000	0	0	0	0	0	0	1	0	0
001	0	1	0	0	0	0	1	1	1
010	0	1	0	0	0	0	1	1	1
011	0	1	0	0	1	1	0	0	0
100	0	1	0	0	0	0	1	1	1
101	0	1	0	1	0	1	0	0	0
110	0	1	1	0	0	1	0	0	0
111	1	1	1	1	1	1	0	0	1

6.4 Two-Level NAND Circuits

Digital circuits are more frequently constructed with NAND and NOR gates than with AND and OR gates. NAND and NOR gates are easier to fabricate with electronic components and are the basic gates in all IC digital logic families. Because of the prominence of NAND and NOR gates in the design of digital circuits, rules and procedures have been developed for the conversion from Boolean functions given in terms of AND, OR, and NOT into equivalent NAND or NOR logic diagrams.

The implementation of a Boolean function with NAND gates requires that the function be simplified in the sum of products form. To see the relationship between a sum of products expression and its equivalent NAND implementation, consider the logic diagram drawn in Figure 6.5. All three diagrams are equivalent and implement the function

$$F = A + BC + CD$$

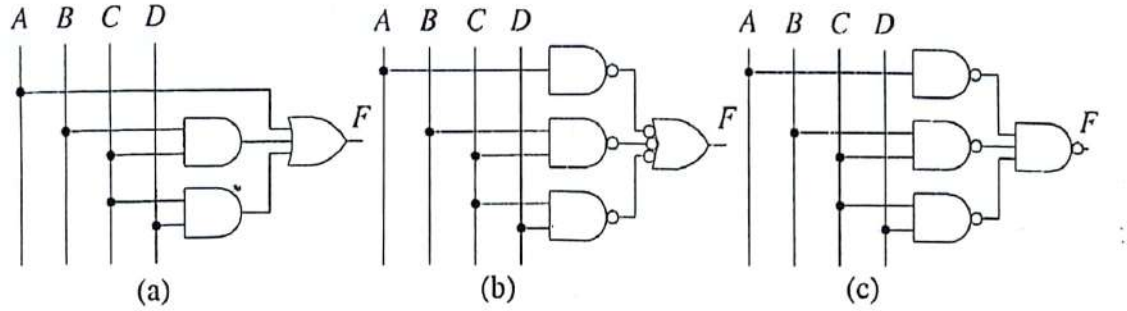


Figure 6.5 Three ways to implement $F = A + BC + CD$.

The function is implemented in Figure 6.5(a) in sum of products form with AND and OR gates. In Figure 6.5(b), the AND gates are replaced by NAND gates and the OR gate is replaced by a NAND gate with an invert-OR symbol (see Figure 4.8). The single variable A is complemented using one-input NAND gate (see Figure 4.8) and applied to the second-level invert-OR gate. Remember that a small circle denotes complementation. Therefore, two circles on the same line represents double complementation and both can be removed. The complement of A goes through a small circle which complements the variable again to produce the uncomplemented value of A. Removing the small circles in the gates of Figure 6.5(b) produces the circuit in Figure 6.5(a). Therefore, the two diagrams implement the same function and are equivalent. In Figure 6.5(c), the output NAND gate is redrawn with the conventional symbol. The diagram in Figure 6.5(c) is equivalent to the one in Figure 6.5(b), which in turn is equivalent to the diagram in Figure 6.5(a). Note the similarity between the diagrams in Figure 6.5(a) and 6.5(c). The AND and OR gates have been changed to NAND gates and an additional one-input NAND gate has been included with the single variable A.

The NAND implementation in Figure 6.5(c) can be verified algebraically using DeMorgan's theorem as follows:

$$F = (A'(BC)'(CD)')' = A + BC + CD$$

From the transformation shown in Figure 6.5, we conclude that a Boolean function can be implemented with two levels of NAND gates. The rule for obtaining the NAND logic diagram from a Boolean function is as follows:

1. Simplify the function and express it in sum of products form.
2. Draw a NAND gate for each product term of the function that has at least two literals. The inputs to each NAND gate are the literals of the product term. This constitutes a group of first-level gates. A term with a single literal requires an inverter in the first level. Therefore, complement a single literal and apply as an input to the second level.
3. Draw a single NAND gate in the second level, with inputs coming from the first level.

The procedure is demonstrated for the following function

$$F(A, B, C, D) = \sum(0,1,2,4,5,6,8,10,12,14,15)$$

Step 1. The simplification of the function as sum of products using K-map is shown in Figure 6.6, which gives

$$F(A,B,C,D) = D' + A'C' + ABC$$

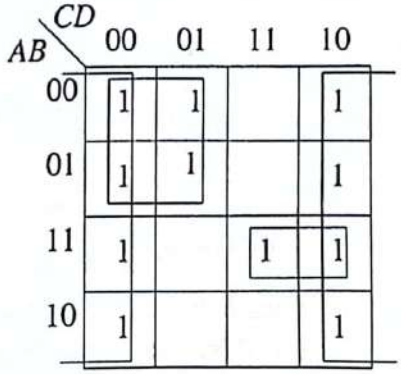


Figure 6.6 Simplification of $F(A,B,C,D) = \sum(0,1,2,4,5,6,8,10,12,14,15)$ in sum of products form using K-map.

Steps 2 and 3. The two-level NAND implementation of the simplified function using steps 2 and 3 is shown in Figure 6.7.

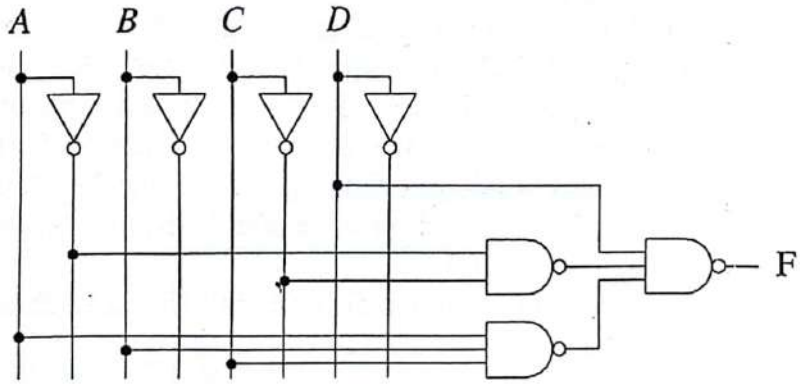


Figure 6.7 Two-level NAND implementation of $F(A,B,C,D) = D' + A'C' + ABC$.

6.5 Two-Level NOR Circuits

The NOR function is the dual of the NAND function. For this reason, all procedures and rules for NOR logic are the dual of the corresponding procedures and rules developed for NAND logic.

The implementation of a Boolean function with NOR gates requires that the function be simplified in product of sums form. A product of sums expression specifies a group of OR gates for the sum terms, followed by an AND gate to produce the product. The transformation from the OR-AND to the NOR-NOR diagram is depicted in Figure 6.8 for the product of sums expression

$$F = A(B + C)(C + D)$$

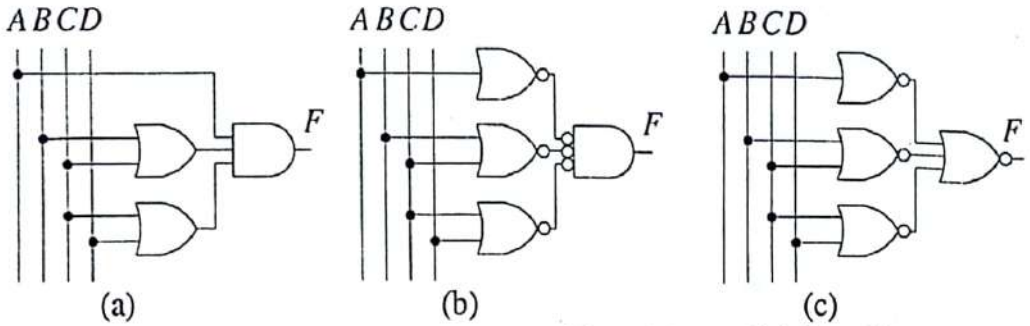


Figure 6.8 Three way to implement $F = A(B + C)(C + D)$.

The rule for obtaining the NOR logic diagram from a Boolean function can be derived from this transformation and is as follows:

1. Simplify the function and express it in product of sums form.
2. Draw a NOR gate for each sum term of the function that has at least two literals. The inputs to each NOR gate are the literals of the sum term. This constitutes a group of first-level gates. A term with a single literal requires an inverter in the first level. Therefore, complement a single literal and apply as an input to the second level.
3. Draw a single NOR gate in the second level, with inputs coming from the first level.

The procedure is demonstrated for the function

$$F(A, B, C, D) = \prod(0,1,4,5,7,8,9,10,11,12,13)$$

Step 1. The simplification of the function as product of sums form using K-map is shown in Figure 6.9, which gives

$$F(A, B, C, D) = C(A' + B)(A + B' + D')$$

AB \ CD	00	01	11	10
00	0	0		
01	0	0	0	
11	0	0		
10	0	0	0	0

Figure 6.9 Simplification of $F(A, B, C, D) = \prod(0,1,4,5,7,8,9,10,11,12,13)$ in product of sums form using K-map.

Steps 2 and 3. The two-level NOR implementation of the simplified function is shown in Figure 6.10.

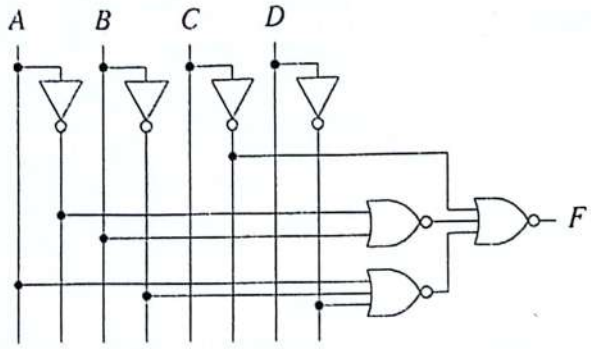


Figure 6.10 Two-level NOR implementation of $F(A, B, C, D) = C(A' + B)(A + B' + D')$.

6.6 Multi-Level NAND Circuits

6.6.1 Boolean Function Implementation with Multi-Level NAND Circuit

The implementation of Boolean functions with multi-level NAND circuits may be obtained by means of a simple block diagram manipulation technique. This method requires that two other logic diagrams be drawn prior to obtaining the NAND logic diagram. Nevertheless, the procedure is very simple and straightforward:

1. From the given algebraic expression, draw the logic diagram with AND, OR, and NOT gates. Assume that both the uncomplemented and complemented inputs are available.
2. Draw a second logic diagram with the equivalent NAND logic, as given in Figure 4.6, substituted for each AND, OR, and NOT gate.
3. Remove any two cascaded inverters from the diagram, since double inversion does not perform a logic function. Remove inverters connected to single external inputs and complement the corresponding input variable. The new logic diagram obtained is the required NAND gate implementation.

This procedure is demonstrated using the following examples.

Example 1: The function is $F = B(C + AD) + CD'$.

1. The AND-OR implementation of this function is shown in the logic diagram of Figure 6.11(a).
2. For each AND gate, we substitute a NAND gate followed by an inverter; for each OR gate, we substitute input inverters followed by a NAND gate. This substitution follows directly from the logic equivalence of Figure 4.6 and is shown in the diagram of Figure 6.11(b). This diagram has seven inverters (excluding the inverters used for indicating the complemented input) and five two-input NAND gates listed with numbers inside the gate symbol.
3. Pairs of inverters connected in cascade (from each AND box to each OR box) are removed since they form double inversion. The inverter connected to input C is removed and the input is connected to C' input. The result is the NAND logic diagram shown in Figure 6.11(c), with the number inside each symbol identifying the gate from Figure 6.11(b).

This example demonstrated that the number of NAND gates required to implement the Boolean function is equal to the number of AND-OR gates, provided both the uncomplemented and the complemented inputs are available.

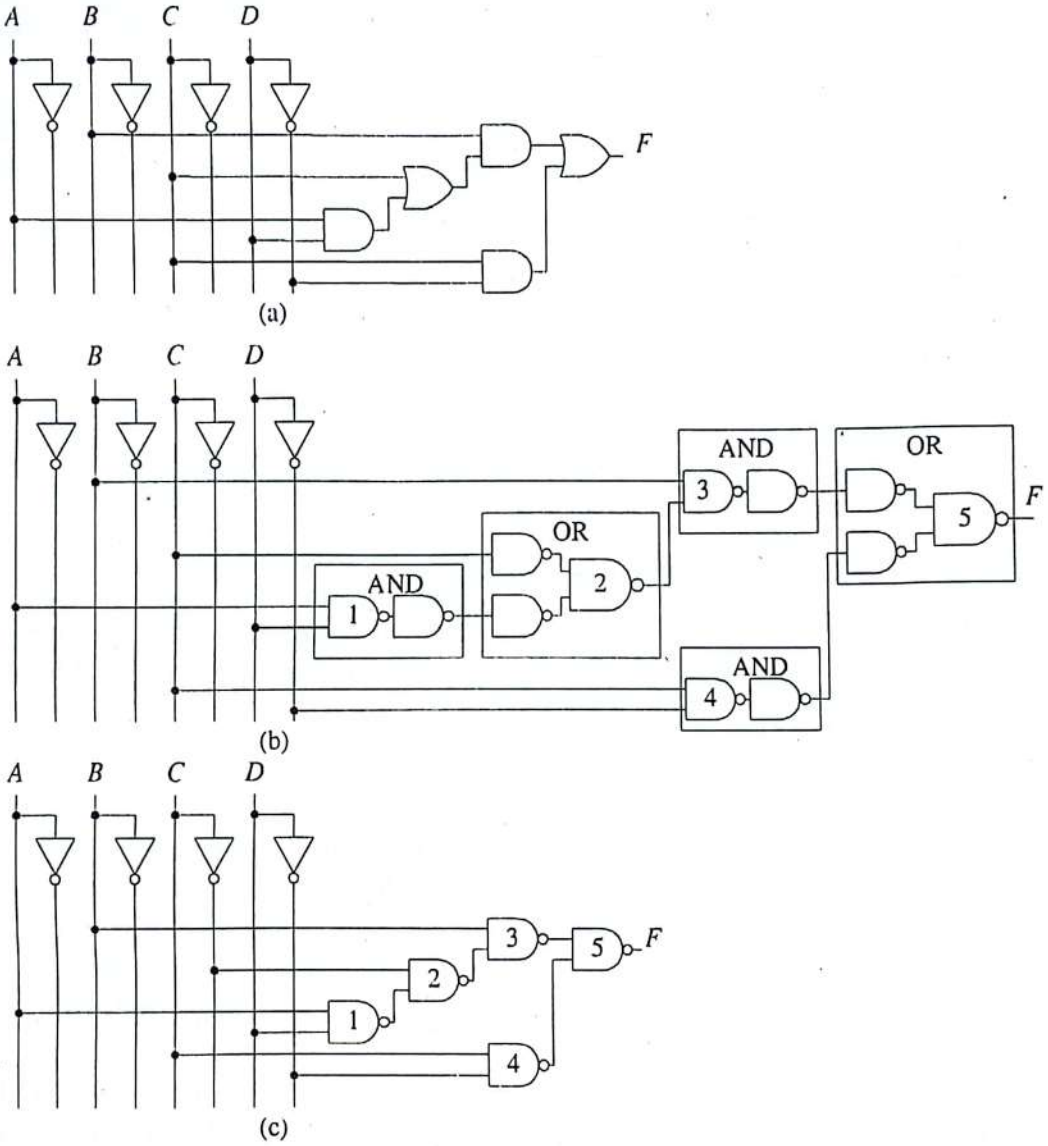


Figure 6.11 Implementation of $F = B(C + AD) + CD'$ with NAND gates.

Example 2: The function is $F = (A' + C)(BC + D)$.

1. The AND-OR implementation is shown in Figure 6.12(a).
2. The NAND logic substitution is shown in Figure 6.12(b).
3. One pair of cascaded inverters from gate 1 to gate 3 is removed. The three external inputs A', C, and D, which directly go to inverters, are complemented and the corresponding inverters are removed. The final NAND gate implementation is shown in Figure 6.12(c).

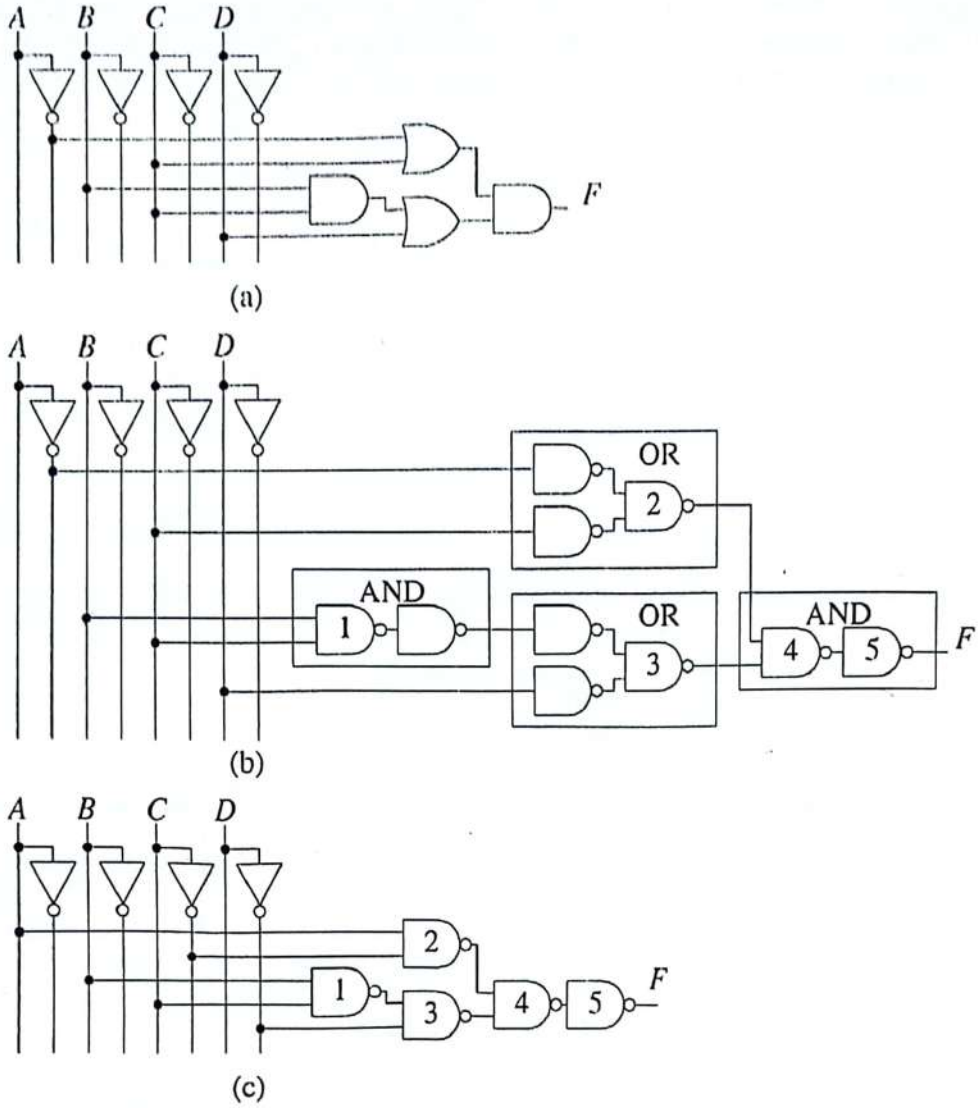


Figure 6.12 Implementation of $F = (A' + C)(BC + D)$ with NAND gates.

The number of NAND gates for this example is equal to the number of AND-OR gates plus an additional inverter in the output (NAND gate 5). In general, the number of NAND gates required to implement a function equals the number of AND-OR gates, except for an occasional inverter. This is true provided both the uncomplemented and the complemented inputs are available.

The block diagram method is somewhat tiresome to use because it requires the drawing of two logic diagrams to obtain the answer in a third.

6.6.2 Analysis of Multi-Level NAND Circuits

The foregoing procedure considered the problem of deriving a NAND logic diagram from a given Boolean function. The reverse process is the analysis problem, which starts with a given NAND logic diagram and culminates with a Boolean expression or a truth table. The analysis of NAND logic diagrams follows the same procedure presented in Section 6.3. The only difference is that NAND logic requires a repeated application of DeMorgan's theorem.

Derivation of the Boolean Expression by Algebraic Manipulation

The procedure for deriving the Boolean expression from a logic diagram is outlined in Section 6.3. This procedure is demonstrated for the NAND logic diagram of Figure 6.11(c), which is redrawn in Figure 6.13.

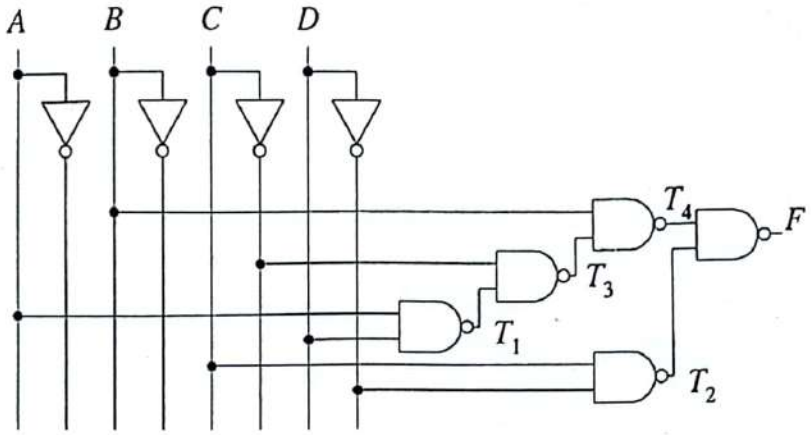


Figure 6.13 Analysis example of NAND circuit.

First, all gate outputs are labeled with arbitrary symbols. Second, the Boolean functions for the outputs of gates that receive only external inputs are derived:

$$T_1 = (AD)' = A' + D'$$
$$T_2 = (CD')' = C' + D$$

The second form follows directly from DeMorgan's theorem and may, at times, be more convenient to use. Third, Boolean functions of gates which have inputs from previously derived functions are determined in consecutive order until the output is expressed in terms of external input variables:

$$T_3 = (C'T_1)' = (A'C' + C'D')' = (A + C)(C + D) = AC + AD + C + CD$$
$$= C + AD$$
$$T_4 = (BT_3)' = (B(C + AD))'$$
$$F = (T_4T_2)' = ((B(C + AD))'(CD')')' = B(C + AD) + CD'$$

Derivation of the Truth Table

The procedure for obtaining the truth table directly from a logic diagram is also outlined in Section 6.3. This procedure is demonstrated for the NAND logic diagram of Figure 6.13. First, the four input variables, together with their 16 combinations of 1s and 0s, are listed in Table 6.3. Second, the outputs of all gates are labeled with arbitrary symbols as in Figure 6.13. Third, we obtain the truth table for the outputs of those gates that are a function of the external input variables only. These are T_1 and T_2. $T_1 = (AD)'$, so we mark 0s in those rows where both A and D are equal to 1 and fill the rest of the rows of T_1 with 1s. Also, $T2 = (CD')'$, so we mark 0s in those rows where $C = 1$ and $D = 0$, and fill the rest of the rows of T_2 with 1s.

We then proceed to obtain the truth table for the outputs of those gates that are a function of previously defined outputs until the column for the output F is determined.

Table 6.3 Truth table for the circuit of Figure 6.13.

A B C D	T_1	T_2	T_3	T_4	F
0 0 0 0	1	1	0	1	0
0 0 0 1	1	1	0	1	0
0 0 1 0	1	0	1	1	1
0 0 1 1	1	1	1	1	0
0 1 0 0	1	1	0	1	0
0 1 0 1	1	1	0	1	0
0 1 1 0	1	0	1	0	1
0 1 1 1	1	1	1	0	1
1 0 0 0	1	1	0	1	0
1 0 0 1	0	1	1	1	0
1 0 1 0	1	0	1	1	1
1 0 1 1	0	1	1	1	0
1 1 0 0	1	1	0	1	0
1 1 0 1	0	1	1	0	1
1 1 1 0	1	0	1	0	1
1 1 1 1	0	1	1	0	1

It is now possible to obtain an algebraic expression for the output from the derived truth table. The K-map shown in Figure 6.14 is obtained directly from Table 6.3. The simplified expression obtained from the K-map is

$$F = BC + ABD + CD' = B(C + AD) + CD'$$

This is the same as the expression of Figure 6.11, thus verifying the correct answer.

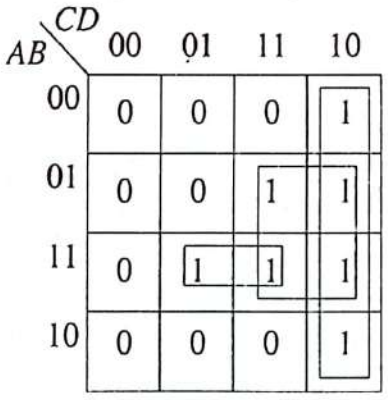


Figure 6.14 Derivation of F from Table 6.3 using K-map.

Conversion of NAND Logic Diagram to AND-OR Logic Diagram

It is sometimes convenient to convert a NAND logic diagram to its equivalent AND-OR logic diagram to facilitate the analysis procedure. By doing so, the Boolean function can be derived more easily without employing DeMorgan's theorem. The conversion of logic diagram is accomplished through a process reverse from that used for implementation. In Figure 4.8, we showed two alternate graphic symbols

for the NAND gate. By judicious use of both symbols, it is possible to convert a NAND diagram to an equivalent AND-OR form.

The conversion of a NAND logic diagram to an AND-OR diagram is achieved through a change in symbols from AND-invert to invert-OR in alternate levels of gates. The first level to be changed to an invert-OR symbol should be the last level. These changes produce pairs of circles along the same line, and these can be removed since they represent double complementation. Moreover, a one-input AND or OR gate can be removed since it does not perform a logical function. A one-input AND or OR with a circle in the input or output is changed to an inverter.

This procedure is demonstrated in Figure 6.15. The NAND logic diagram of Figure 6.15(a) is that of Figure 6.11(c) and is to be converted to an AND-OR logic diagram. The symbol of the gate in the last level is changed to an invert-OR. Looking for alternate levels, we find one gate requiring a change of symbol as shown in Figure 6.15(b). Any two circles along the same line are removed. Circles that go to external inputs are also removed and the corresponding input variable is complemented. The required AND-OR logic diagram is drawn in Figure 6.15(c).

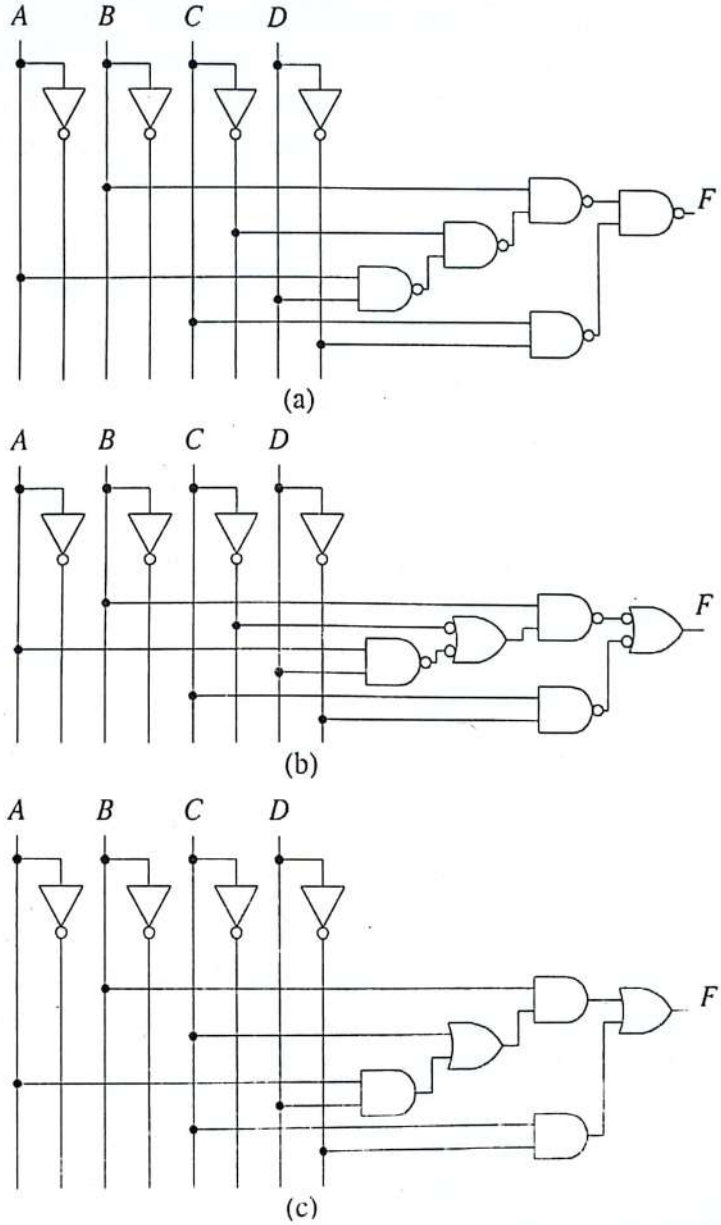


Figure 6.15 Conversion of NAND logic diagram to AND-OR logic diagram